Actividad autónoma 11

Pruebas unitarias y escenarios offline

Estudiante: Grease Duran Castro

Desarrollo de Aplicaciones Móviles | Ciclo VI

Fecha: 20 de octubre de 2026

Proyecto: PharmaMobil sobre PharmaSoft

Rama: feature/autonoma11-duran

https://github.com/GreysiLL/AP11/tree/feature/autonoma11-duran

Propósito y resultado

Se amplió PharmaMobil con la antigüedad de los datos, un estado vacío para la primera apertura sin conexión y consulta de la caché en modo lectura cuando una sesión previamente validada vence sin red. La entrega reúne los cuatro productos de la guía: seis pruebas, cinco escenarios con evidencia real, antigüedad y guion de sustentación.

Comprobación	Resultado
Suite de la guía	6 de 6 pruebas aprobadas
Regresión del proyecto	102 pruebas aprobadas, sin fallos, errores u omisiones
Bitácora Android	Cinco escenarios con captura y estado local
Producto creado sin red	AA11-1020151700: ID local -1, después ID remoto 83
Antigüedad	Nunca sincronizado, reciente y dos días atrás
iOS	Implementación preparada; ejecución pendiente por falta de Mac

La instalación de pruebas usa el paquete aislado pe.edu.upeu.pharmamobil.aa11. El borrado de datos y las fixtures no afectaron la instalación habitual. Las capturas pertenecen a Android 16 API 36, Pixel 9 (2). Las horas de la bitácora corresponden a Lima; el reloj del emulador usa otra zona horaria.

Producto 1 Suite de seis pruebas

Archivo: shared/src/commonTest/kotlin/pe/edu/upeu/pharmamobil/data/repository/Autonoma11SuiteTest.kt. Cada prueba construye su propio driver en memoria y un MockEngine. No necesita emulador, internet ni esperas reales.

Caso	Verificación y resultado
T01 Sincronización con red	Descarga dos páginas, conserva ambos IDs y el Flow emite la lista completa. Aprobado.
T02 Registro sin red	El envío falla, pero SQLite conserva ID negativo, pendiente 1 y el Flow emite el producto. Aprobado.
T03 Envío al reconectar	POST ocurre antes de GET; el servidor asigna ID 70 y la marca pendiente desaparece. Aprobado.
T04 La bajada no borra pendientes	La descarga simulada se aplica a la capa local mientras existe una fila pendiente; ambas se observan desde el repositorio. Aprobado.
T05 Identificadores sin colisión	Conviven el ID local -1 y el remoto 1 sin sobrescribir nombre ni contenido. Aprobado.
T06 Dos sincronizaciones no duplican	La segunda sincronización conserva la misma lista y solo existe un POST de creación. Aprobado.

T02 y T04 se escribieron primero. T04 aísla la fase de descarga para probar su protección: la sincronización completa correctamente no descarga si antes falla una subida. No se invierte ese orden para hacer pasar la prueba.

Se conservaron las pruebas anteriores y se verificaron además la marca de sincronización ante errores, la migración del esquema 1 a 2 sin borrar productos, el texto de antigüedad y el modo lectura de la sesión vencida. La ejecución completa del proyecto aprobó 102 pruebas; la ejecución específica aprobó los seis casos de esta suite.

El anexo A incluye la salida completa de Gradle para los seis casos. El anexo B incluye el código de la suite. En el repositorio también se conserva el log íntegro de las 102 pruebas y el XML JUnit de la suite.

Producto 2 Bitácora E01

Apertura sin red con datos previos

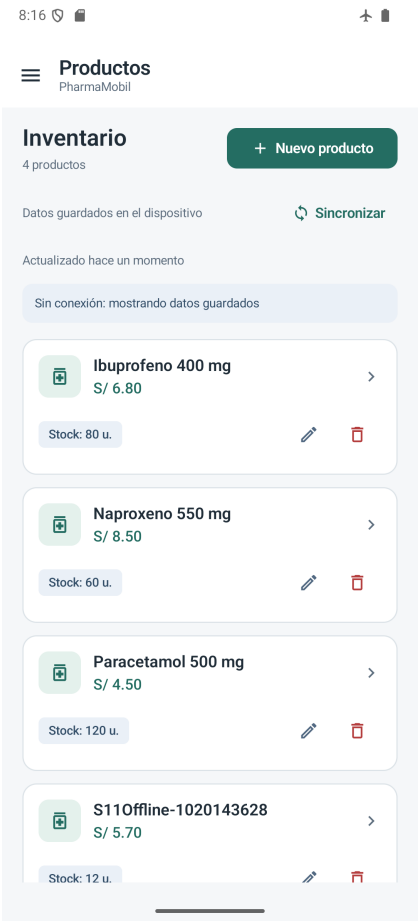
Fecha y hora: 2026-10-20T15:16:40.205120-05:00 | Plataforma: Android

Pasos: Iniciar sesión con admin y sincronizar. Cerrar por completo la app, activar modo avión, desactivar Wi-Fi y datos, y reabrir.

Resultado observado: El inventario muestra los cuatro productos guardados, el aviso sin conexión y la antigüedad de la última sincronización.

Base local: Cuatro filas sincronizadas, sin pendientes; se conserva la marca de la última sincronización correcta.

Conclusión: La lectura depende de SQLite y no de que PharmaSoft responda.



Captura E01 del proyecto en ejecución.

Producto 2 Bitácora E02

Registro sin red

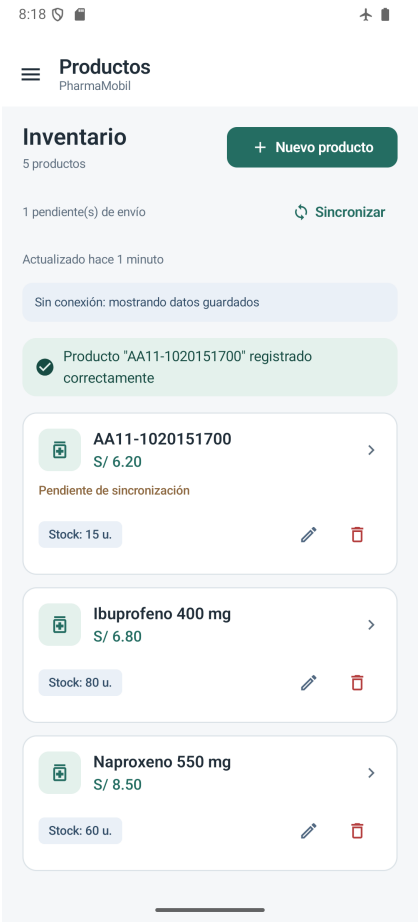
Fecha y hora: 2026-10-20T15:18:24.420371-05:00 | Plataforma: Android

Pasos: En modo avión registrar AA11-1020151700, precio 6.20 y stock 15. Revisar la fila y el contador de pendientes.

Resultado observado: La lista incorpora el producto y lo identifica como pendiente; el listado no se pierde.

Base local: Cinco filas: cuatro sincronizadas y una con id -1, pendiente 1, precio 6.20 y stock 15.

Conclusión: El registro se conserva localmente aunque el envío no pueda completarse.



Captura E02 del proyecto en ejecución.

Producto 2 Bitácora E03

Reconexión y envío

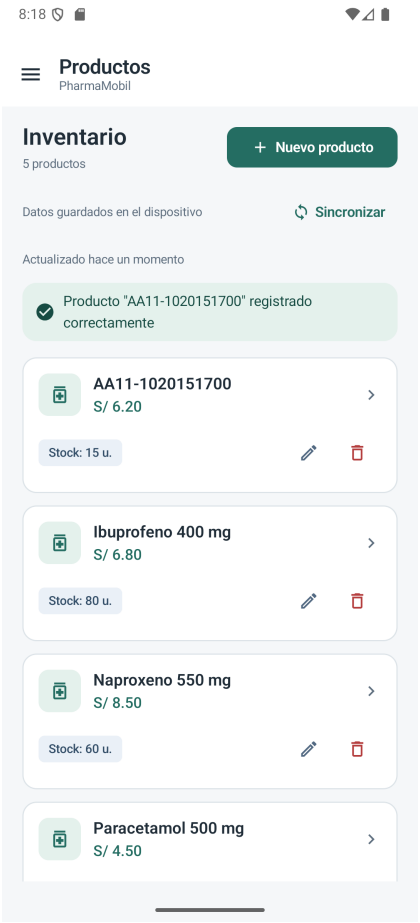
Fecha y hora: 2026-10-20T15:18:46.015150-05:00 | Plataforma: Android

Pasos: Desactivar modo avión, habilitar Wi-Fi y datos y pulsar Sincronizar. Consultar el producto por ID en Swagger con autorización de admin.

Resultado observado: El pendiente desaparece y Swagger devuelve 200 para el producto con ID 83.

Base local: La fila negativa se sustituye por id 83, pendiente 0. Hay cinco filas y solo una corresponde al producto de prueba.

Conclusión: La comprobación del servidor acredita que el dato salió del dispositivo, sin duplicarse.



Captura E03 del proyecto en ejecución.

E03 Comprobación adicional en Swagger

GET /api/v1/productos/83 respondió 200 con AA11-1020151700, precio 6.20, stock 15 y estado activo. La fila local quedó con ID 83 y pendiente 0; se conserva una sola fila de este producto. La respuesta prueba que llegó a PharmaSoft.



La captura encuadra URL y respuesta sin mostrar el JWT de la sección curl.

Preparación del servidor: Oracle Docker y PharmaSoft estaban encendidos. La petición usó autenticación de admin y el producto exclusivo de prueba se conserva activo.

Producto 2 Bitácora E04

Primera apertura sin red

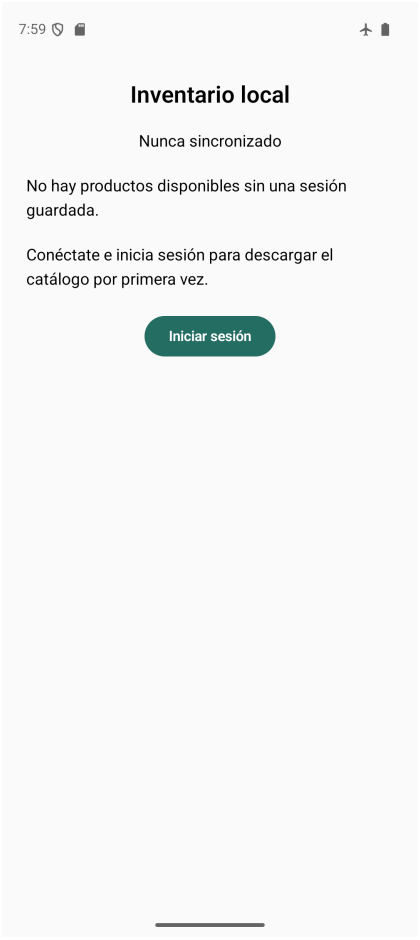
Fecha y hora: 2026-10-20T14:59:02.501155-05:00 | Plataforma: Android

Pasos: Borrar datos exclusivamente del paquete aislado pe.edu.upeu.pharmamobil.aa11. Activar modo avión y abrir. Desde el login pulsar Ver catálogo sin conexión.

Resultado observado: Se muestra Inventario local, Nunca sincronizado y la explicación de que hace falta iniciar sesión con red para descargar el catálogo.

Base local: Sin productos ni marca de sincronización. La inspección run-as confirmó que pharma.db todavía no estaba creado: el driver abre SQLite de forma diferida al ejecutar la primera consulta.

Conclusión: Sin una cuenta validada ni un catálogo descargado corresponde un estado vacío informativo; no se inventan datos ni se expone la caché de otra cuenta.



Captura E04 del proyecto en ejecución.

Producto 2 Bitácora E05

Sesión expirada sin red

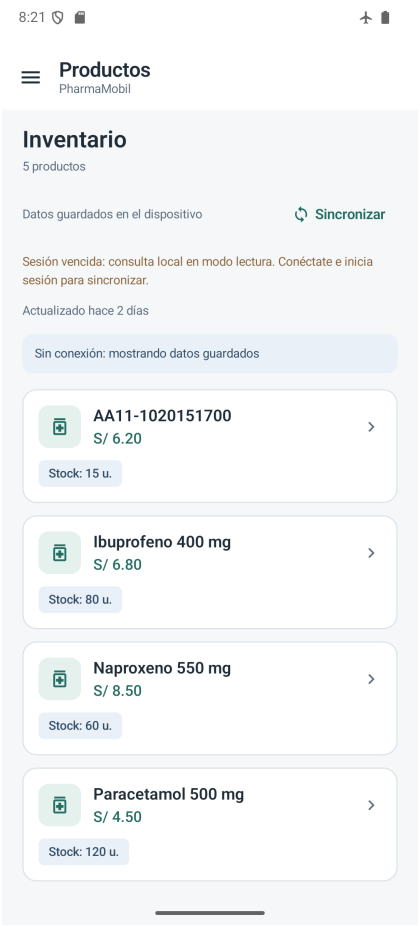
Fecha y hora: 2026-10-20T15:21:10.862535-05:00 | Plataforma: Android

Pasos: Con admin validado y cinco productos locales, cargar en la instalación aislada un JWT de prueba firmado con exp de hace una hora. Confirmar previamente 401 en auth/me, activar modo avión y reabrir. Después reconectar y pulsar Sincronizar.

Resultado observado: Sin red se muestra el inventario en modo lectura, con aviso de sesión vencida; no hay Nuevo producto ni acciones de edición o eliminación. Al reconectar, el 401 devuelve al login. Tras iniciar sesión válidamente se recupera el uso normal.

Base local: Las cinco filas y la marca de sincronización se conservan tanto durante la consulta local como después del 401.

Conclusión: Es preferible permitir la consulta de datos ya guardados con un aviso claro a exigir un login imposible sin red. Se bloquean escrituras y el servidor decide al reconectar; no se renueva ni se valida la firma del JWT en el cliente.



Captura E05 del proyecto en ejecución.

El JWT de prueba estaba firmado con la clave de desarrollo local y su campo exp correspondía a una hora antes. Se confirmó 401 en auth/me antes de cargarlo; no fue simplemente una cadena inválida. No se publican la clave ni el JWT.

E05 Resolución de la sesión al reconectar

Al recuperar la conexión y pulsar Sincronizar, PharmaSoft rechazó el JWT con 401. La aplicación limpió la sesión, mostró el aviso y volvió al login, conservando las cinco filas locales. Luego se inició sesión correctamente y se sincronizó de nuevo.

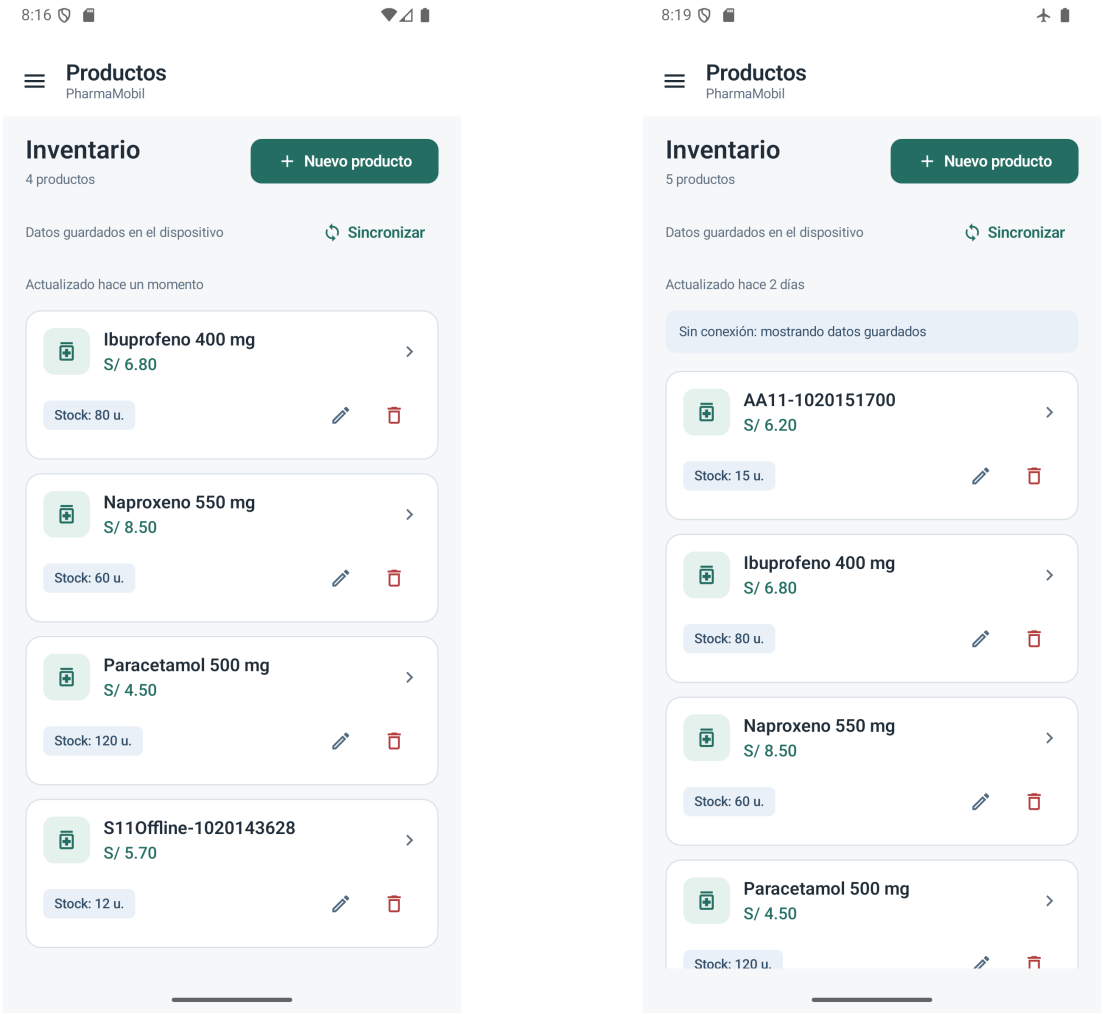


Análisis: Consultar datos previamente descargados es útil cuando no existe red para iniciar sesión. La app informa que la sesión venció, oculta escrituras y deja al servidor decidir al reconectar. Esto no convierte un JWT vencido en válido ni permite acceso remoto sin autorización.

Producto 3 Antigüedad de los datos

Sincronizacion.sq guarda el último instante correcto por usuario. La migración 1.sqm agrega esa tabla sin borrar el catálogo. El repositorio escribe la marca dentro de la transacción de descarga, después de enviar pendientes y completar todas las páginas. Si falla el proceso se conserva la marca anterior.

El ViewModel observa el dato y la pantalla presenta minutos, horas o días en lenguaje natural. Nunca sincronizado es el estado explícito cuando no existe una marca. La interfaz refresca la frase cada minuto.



Izquierda: sincronización reciente real. Derecha: Actualizado hace 2 días con una marca aplicada mediante la fixture de debug en la base aislada. La imagen procede de la app real; no se editó la captura ni se afirma haber esperado dos días. Después de sincronizar correctamente la marca vuelve a ser reciente.

Producto 4 Guion de ocho minutos

Preparación: Mantener Oracle Docker y PharmaSoft encendidos, APK compilado y catálogo precargado. Dejar la sesión cerrada. Tener Swagger abierto con los logins admin y almacén preparados, productos DemoU2 y DemoOffline con nombres únicos, y los ajustes de modo avión accesibles. Usar un emulador que ya haya terminado de arrancar.

Minutos y criterio	Usuario y demostración
0 a 1 | Autenticación JWT	admin / Admin123. Mostrar un login incorrecto, corregir la clave y entrar al inventario. Explicar que Ktor obtiene y adjunta el JWT y que el almacén lo cifra.
1 a 3 | REST y CRUD completo	admin. Listar, registrar DemoU2 con precio 4.50 y stock 12, editar precio a 5.20 y eliminar solo ese producto. Contrastar la petición y la respuesta del servidor.
3 a 5 | Estados y errores	almacen / Almacen123. Cerrar sesión, entrar con almacén, enviar el formulario vacío y señalar los errores bajo los campos. Mostrar GET clientes en Swagger con el token de almacén: 403; la sesión de la app se mantiene.
5 a 7 | Persistencia offline-first	almacen. Modo avión y Wi-Fi/datos apagados: reabrir, mostrar catálogo y antigüedad, crear DemoOffline pendiente, reconectar, pulsar Sincronizar y comprobar el ID positivo en Swagger.
7 a 8 | Contraste por plataforma	Misma cuenta. Mostrar la ejecución Android y ubicar los drivers Android/iOS y el código común. La ejecución iOS está pendiente por falta de Mac; explicar esta limitación y no presentar código como si fuera una ejecución.

Control del tiempo: A los minutos 1, 3, 5 y 7 pasar al tramo siguiente. Preparar las peticiones y los nombres antes de comenzar; no compilar ni iniciar el emulador durante la exposición. Ensayar en voz alta con cronómetro antes de sustentar.

Límite multiplataforma: El último minuto cubre el contraste de arquitectura y la ejecución Android. La demostración iOS real sigue pendiente por falta de Mac/Xcode; mostrar los drivers no reemplaza esa evidencia. No se atribuye a iOS ninguna captura de este documento.

Cambios registrados

aadca12: antigüedad persistida, consulta local con sesión vencida y estado vacío. fa80973: suite de seis casos, regresiones y fixtures aisladas. La documentación, las capturas y este PDF se guardan en la misma rama de entrega.

Anexo A Salida completa de la suite

Comando ejecutado: gradlew.bat :androidApp:assembleDebug :shared:testAndroidHostTest --tests *Autonoma11SuiteTest --no-daemon --no-configuration-cache --console=plain -Paa11Pruebas=true. Se usaron además opciones de compilación y firma debug locales. La salida siguiente está completa; solo se ajustan saltos de línea para que quepa en la página.

```
To honour the JVM settings for this build a single-use Daemon process will be forked. For more on this,
please refer to https://docs.gradle.org/9.1.0/userguide/gradle_daemon.html#sec:disabling_the_daemon in the
Gradle documentation.
Daemon JVM discovery is an incubating feature.
Daemon will be stopped at the end of the build

> Configure project :shared
w: Native task 'iosSimulatorArm64Test' is disabled
Task 'iosSimulatorArm64Test' for target 'ios_simulator_arm64' cannot run on the current host
(windows-x86_64).
Reason: simulator tests require macOS
Solution: To suppress this warning, add 'kotlin.native.ignoreDisabledTargets=true' to gradle.properties.


> Task :androidApp:preBuild UP-TO-DATE
> Task :androidApp:preDebugBuild UP-TO-DATE
> Task :androidApp:mergeDebugNativeDebugMetadata NO-SOURCE
> Task :androidApp:generateDebugResources UP-TO-DATE
> Task :androidApp:packageDebugResources UP-TO-DATE
> Task :shared:androidPreBuild UP-TO-DATE
> Task :shared:preAndroidMainBuild UP-TO-DATE
> Task :shared:generateAndroidMainEmptyResourceFiles UP-TO-DATE
> Task :androidApp:processDebugNavigationResources UP-TO-DATE
> Task :androidApp:parseDebugLocalResources UP-TO-DATE
> Task :androidApp:generateDebugRFile UP-TO-DATE
> Task :shared:kmpPartiallyResolvedDependenciesChecker
> Task :shared:checkKotlinGradlePluginConfigurationErrors SKIPPED
> Task :shared:convertXmlValueResourcesForAndroidMain NO-SOURCE
> Task :shared:copyNonXmlValueResourcesForAndroidMain NO-SOURCE
> Task :shared:prepareComposeResourcesTaskForAndroidMain NO-SOURCE
> Task :shared:generateResourceAccessorsForAndroidMain NO-SOURCE
> Task :shared:convertXmlValueResourcesForCommonMain NO-SOURCE
> Task :shared:copyNonXmlValueResourcesForCommonMain UP-TO-DATE
> Task :shared:prepareComposeResourcesTaskForCommonMain UP-TO-DATE
> Task :shared:generateResourceAccessorsForCommonMain UP-TO-DATE
> Task :shared:generateActualResourceCollectorsForAndroidMain UP-TO-DATE
> Task :shared:copyAndroidMainComposeResourcesToAndroidAssets UP-TO-DATE
> Task :shared:packageAndroidMainResources UP-TO-DATE
> Task :shared:parseAndroidMainLocalResources UP-TO-DATE
> Task :shared:generateAndroidMainRFile UP-TO-DATE
> Task :shared:generateCommonMainPharmaDbInterface UP-TO-DATE
> Task :shared:generateComposeResClass UP-TO-DATE
> Task :shared:generateExpectResourceCollectorsForCommonMain UP-TO-DATE
> Task :androidApp:javaPreCompileDebug UP-TO-DATE
> Task :androidApp:generateDebugAssets UP-TO-DATE
> Task :shared:generateAndroidMainAssets UP-TO-DATE
> Task :shared:mergeAndroidMainAssets UP-TO-DATE
> Task :androidApp:mergeDebugAssets UP-TO-DATE
> Task :androidApp:compressDebugAssets UP-TO-DATE
> Task :androidApp:checkDebugDuplicateClasses UP-TO-DATE
> Task :androidApp:desugarDebugFileDependencies UP-TO-DATE
> Task :androidApp:mergeExtDexDebug UP-TO-DATE
> Task :shared:writeAndroidMainAarMetadata UP-TO-DATE
> Task :androidApp:checkDebugAarMetadata UP-TO-DATE
> Task :androidApp:compileDebugNavigationResources UP-TO-DATE
> Task :androidApp:mapDebugSourceSetPaths UP-TO-DATE
> Task :androidApp:mergeDebugResources UP-TO-DATE
> Task :androidApp:createDebugCompatibleScreenManifests UP-TO-DATE
> Task :androidApp:extractDeepLinksDebug UP-TO-DATE
> Task :shared:processAndroidMainManifest UP-TO-DATE
> Task :androidApp:processDebugMainManifest UP-TO-DATE
> Task :androidApp:processDebugManifest UP-TO-DATE
> Task :androidApp:processDebugManifestForPackage UP-TO-DATE
> Task :shared:compileAndroidMainLibraryResources UP-TO-DATE
> Task :androidApp:processDebugResources UP-TO-DATE
> Task :androidApp:mergeDebugJniLibFolders UP-TO-DATE
> Task :shared:mergeAndroidMainJniLibFolders UP-TO-DATE
> Task :shared:mergeAndroidMainNativeLibs NO-SOURCE
> Task :shared:copyAndroidMainJniLibsProjectOnly UP-TO-DATE
> Task :androidApp:mergeDebugNativeLibs UP-TO-DATE
> Task :androidApp:stripDebugDebugSymbols UP-TO-DATE
> Task :androidApp:validateSigningDebug UP-TO-DATE
> Task :androidApp:writeDebugAppMetadata UP-TO-DATE
> Task :androidApp:writeDebugSigningConfigVersions UP-TO-DATE
> Task :shared:androidJar UP-TO-DATE
> Task :shared:generateAndroidHostTestAssets UP-TO-DATE
> Task :shared:preAndroidHostTestBuild UP-TO-DATE
```

```
> Task :shared:mergeAndroidHostTestAssets UP-TO-DATE
> Task :shared:checkAndroidHostTestAarMetadata UP-TO-DATE
> Task :shared:mapAndroidHostTestSourceSetPaths UP-TO-DATE
> Task :shared:generateAndroidHostTestResources UP-TO-DATE
> Task :shared:mergeAndroidHostTestResources UP-TO-DATE
> Task :shared:processAndroidHostTestManifest UP-TO-DATE
> Task :shared:processAndroidHostTestResources UP-TO-DATE
> Task :shared:packageAndroidHostTestForUnitTest UP-TO-DATE
> Task :shared:generateAndroidHostTestConfig UP-TO-DATE
> Task :shared:convertXmlValueResourcesForAndroidHostTest NO-SOURCE
> Task :shared:copyNonXmlValueResourcesForAndroidHostTest NO-SOURCE
> Task :shared:prepareComposeResourcesTaskForAndroidHostTest NO-SOURCE
> Task :shared:generateResourceAccessorsForAndroidHostTest NO-SOURCE
> Task :shared:convertXmlValueResourcesForCommonTest NO-SOURCE
> Task :shared:copyNonXmlValueResourcesForCommonTest NO-SOURCE
> Task :shared:prepareComposeResourcesTaskForCommonTest NO-SOURCE
> Task :shared:generateResourceAccessorsForCommonTest NO-SOURCE
> Task :shared:processAndroidHostTestJavaRes NO-SOURCE
> Task :shared:compileAndroidMain
w: file:///C:/Users/HECTOR/StudioProjects/PHARMAMOBILfijo/shared/src/androidMain/kotlin/pe/edu/upeu/pharmamo
bil/platform/Formato.android.kt:5:38 'constructor(p0: String!, p1: String!): Locale' is deprecated.
Deprecated in Java.
w: file:///C:/Users/HECTOR/StudioProjects/PHARMAMOBILfijo/shared/src/androidMain/kotlin/pe/edu/upeu/pharmamo
bil/platform/InfoDispositivo.android.kt:5:1 'expect'/'actual' classes (including interfaces, objects,
annotations, enums, and 'actual' typealiases) are in Beta. Consider using the '-Xexpect-actual-classes' flag
 to suppress this warning. Also see: https://youtrack.jetbrains.com/issue/KT-61573
w: file:///C:/Users/HECTOR/StudioProjects/PHARMAMOBILfijo/shared/src/commonMain/kotlin/pe/edu/upeu/pharmamob
il/App.kt:98:13 'fun KoinContext(koin: Koin = ..., content: ComposableFunction0<Unit>): Unit' is deprecated.
 KoinContext is not needed anymore. This can be removed. Compose Koin context is setup with StartKoin().
w: file:///C:/Users/HECTOR/StudioProjects/PHARMAMOBILfijo/shared/src/commonMain/kotlin/pe/edu/upeu/pharmamob
il/data/remote/EjecutarLlamada.kt:20:149 Unnecessary non-null assertion (!!) on a non-null receiver of type
'ErrorResponseDto'.
w: file:///C:/Users/HECTOR/StudioProjects/PHARMAMOBILfijo/shared/src/commonMain/kotlin/pe/edu/upeu/pharmamob
il/platform/InfoDispositivo.kt:3:1 'expect'/'actual' classes (including interfaces, objects, annotations,
enums, and 'actual' typealiases) are in Beta. Consider using the '-Xexpect-actual-classes' flag to suppress
this warning. Also see: https://youtrack.jetbrains.com/issue/KT-61573
w: file:///C:/Users/HECTOR/StudioProjects/PHARMAMOBILfijo/shared/src/commonMain/kotlin/pe/edu/upeu/pharmamob
il/presentation/detalle/DetalleProductoScreen.kt:17:69 'val Icons.Outlined.ArrowBack: ImageVector' is
deprecated. Use the AutoMirrored version at Icons.AutoMirrored.Outlined.ArrowBack.

> Task :shared:processAndroidMainJavaRes UP-TO-DATE
> Task :shared:bundleLibRuntimeToDirAndroidMain
> Task :shared:bundleAndroidMainClassesToRuntimeJar
> Task :shared:bundleAndroidMainClassesToCompileJar
> Task :androidApp:compileDebugKotlin UP-TO-DATE
> Task :androidApp:compileDebugJavaWithJavac NO-SOURCE
> Task :androidApp:processDebugJavaRes UP-TO-DATE
> Task :androidApp:mergeDebugJavaResource UP-TO-DATE
> Task :androidApp:dexBuilderDebug UP-TO-DATE
> Task :androidApp:mergeProjectDexDebug UP-TO-DATE
> Task :shared:compileAndroidHostTest UP-TO-DATE

> Task :shared:testAndroidHostTest

Autonoma11SuiteTest > T05_identificadoresLocalesYRemotosNoColisionan STANDARD_ERROR
    SLF4J(W): No SLF4J providers were found.
    SLF4J(W): Defaulting to no-operation (NOP) logger implementation
    SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further details.

Autonoma11SuiteTest > T05_identificadoresLocalesYRemotosNoColisionan PASSED

Autonoma11SuiteTest > T01_sincronizacionConRedEmiteTodasLasPaginas PASSED

Autonoma11SuiteTest > T03_reconexionEnviaYPoneLaMarcaEnCero PASSED

Autonoma11SuiteTest > T02_registroSinRedQuedaPendienteYEmiteElFlow PASSED

Autonoma11SuiteTest > T06_sincronizarDosVecesNoDuplicaFilasNiPost PASSED

Autonoma11SuiteTest > T04_laBajadaConservaLoPendiente PASSED

> Task :androidApp:mergeLibDexDebug
> Task :androidApp:packageDebug
> Task :androidApp:createDebugApkListingFileRedirect UP-TO-DATE
> Task :androidApp:assembleDebug

[Incubating] Problems report is available at:
file:///C:/Users/HECTOR/StudioProjects/PHARMAMOBILfijo/build/reports/problems/problems-report.html

BUILD SUCCESSFUL in 1m 11s
71 actionable tasks: 8 executed, 63 up-to-date
```

Anexo B Código de las seis pruebas

```
package pe.edu.upeu.pharmamobil.data.repository
import app.cash.sqldelight.db.SqlDriver
import io.ktor.client.HttpClient
import io.ktor.client.engine.mock.*
import io.ktor.http.*
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.test.*
import pe.edu.upeu.pharmamobil.data.local.*
import pe.edu.upeu.pharmamobil.data.remote.*
import pe.edu.upeu.pharmamobil.data.mapper.toDomain
import pe.edu.upeu.pharmamobil.db.PharmaDb
import pe.edu.upeu.pharmamobil.domain.model.*
import kotlin.test.*

/** Seis casos de la guía. Cada fixture crea su propio driver; MockEngine no abre la red. */
@OptIn(kotlinx.coroutines.ExperimentalCoroutinesApi::class)
class Autonoma11SuiteTest {
 private val drivers=mutableListOf<SqlDriver>();private val clientes=mutableListOf<HttpClient>()
 private val sesion=MutableStateFlow<Sesion?>(Sesion("prueba","admin","Admin",setOf("CATALOGO_LEER","CATALOG
O_ESCRIBIR")))
 private fun producto(id:Long=10,nombre:String="Servidor")="""{"id":$id,"nombre":"$nombre","precio":2.5,"sto
ck":12,"estado":true,"categoriaId":1}"""
 private fun pagina(filas:String,ultima:Boolean=true)="""{"contenido":[$filas],"pagina":0,"tamanio":100,"tot
alElementos":2,"totalPaginas":2,"ultima":$ultima}"""
 private fun MockRequestHandleScope.json(cuerpo:String,status:HttpStatusCode=HttpStatusCode.OK)=respond(cuer
po,status,headersOf(HttpHeaders.ContentType,"application/json"))
 private fun
fixture(scope:TestScope,engine:MockEngine):Triple<ProductoRepositorioOffline,ProductoLocal,ProductoApi> {
  val driver=driverEnMemoria().also(drivers::add);val
local=ProductoLocal(PharmaDb(driver),UnconfinedTestDispatcher(scope.testScheduler))
  val cliente=crearHttpClient(engine,"http://localhost/api/v1/",esDepuracion=false).also(clientes::add)
  val api=ProductoApi(cliente)
  return
Triple(ProductoRepositorioOffline(local,api,sesion,UnconfinedTestDispatcher(scope.testScheduler)),local,api)
 }
 @AfterTest fun cerrar() { clientes.forEach { it.close() };drivers.forEach { it.close() } }

 @Test fun T02_registroSinRedQuedaPendienteYEmiteElFlow()=runTest {
  val (repo,local)=fixture(this,MockEngine { throw kotlinx.io.IOException("sin red") })
  val p=repo.registrar(Producto(0,"Producto offline",3.0,9))
  assertTrue(p.id<0);assertTrue(p.pendiente)
  assertEquals(p,local.pendientes("admin").single());assertEquals(listOf(p),repo.observar().first())
 }
 @Test fun T04_laBajadaConservaLoPendiente()=runTest {
  val (repo,local,api)=fixture(this,MockEngine { json(pagina(producto())) })
  val pendiente=local.guardarPendiente("admin",Producto(0,"No perder",3.0,9))
  // Aísla la fase de descarga: sincronizar() no baja si antes falla una subida.
  val descargados=api.listar().contenido.map { it.toDomain() }
  local.reemplazarSincronizados("admin",descargados)
  val visibles=repo.observar().first()
  assertEquals(2,visibles.size);assertEquals(pendiente,visibles.single { it.id<0 })
  assertTrue(local.pendientes("admin").single().pendiente)
 }
 @Test fun T01_sincronizacionConRedEmiteTodasLasPaginas()=runTest {
  val (repo,local)=fixture(this,MockEngine { req ->
   if(req.url.parameters["pagina"]=="0") json(pagina(producto(10),false))
   else json(pagina(producto(11,"Segunda página")))
  })
  repo.sincronizar()
  assertEquals(setOf(10L,11L),repo.observar().first().map { it.id }.toSet())
  assertEquals(2,local.listar("admin").size);assertTrue(local.pendientes("admin").isEmpty())
 }
 @Test fun T03_reconexionEnviaYPoneLaMarcaEnCero()=runTest {
  var red=false;val metodos=mutableListOf<HttpMethod>()
  val (repo,local)=fixture(this,MockEngine { req ->
   if(!red) throw kotlinx.io.IOException("sin red")
   metodos.add(req.method)
   if(req.method==HttpMethod.Post) json(producto(70,"Nuevo offline"),HttpStatusCode.Created)
   else json(pagina(producto(70,"Nuevo offline")))
  })
  repo.registrar(Producto(0,"Nuevo offline",2.5,12));red=true;repo.sincronizar()
  assertEquals(listOf(HttpMethod.Post,HttpMethod.Get),metodos)
  assertTrue(local.pendientes("admin").isEmpty())
  assertEquals(70L,repo.observar().first().single().id);assertFalse(repo.observar().first().single().pendien
te)
 }
 @Test fun T05_identificadoresLocalesYRemotosNoColisionan()=runTest {
  val (repo,local)=fixture(this,MockEngine { json(pagina(producto(1))) })
  local.reemplazarSincronizados("admin",listOf(Producto(1,"Servidor",2.5,12)))
  val nuevo=local.guardarPendiente("admin",Producto(0,"Local",3.0,9))
  assertEquals(-1L,nuevo.id)
  assertEquals(setOf(-1L,1L),repo.observar().first().map { it.id }.toSet())
  assertEquals("Servidor",local.obtener("admin",1).nombre)
 }
```

```
 @Test fun T06_sincronizarDosVecesNoDuplicaFilasNiPost()=runTest {
  var posts=0
  val (repo,local)=fixture(this,MockEngine { req ->
   if(req.method==HttpMethod.Post) { posts++;json(producto(22),HttpStatusCode.Created) }
   else json(pagina(producto(22)))
  })
  local.guardarPendiente("admin",Producto(0,"Servidor",2.5,12))
  repo.sincronizar();val primera=repo.observar().first()
  repo.sincronizar();val segunda=repo.observar().first()
  assertEquals(primera,segunda);assertEquals(1,segunda.size);assertEquals(1,posts)
 }
}
```